

UniVoice — Core API Backend 설계

관리 서버: 코드 구조 / 도메인 모델 / 핵심 흐름 · 졸업프로젝트 코드리뷰용 (2026-05-26)

0. 먼저 경계부터: Core API가 하는 일 / 안 하는 일

PDF에 명시된 가장 중요한 원칙입니다.

"음성 프록시 역할은 하지 않음 — 세션·권한·메타데이터 관리만 담당"

한다 (Core API)	안 한다 (다른 서버)
로그인/JWT 인증	음성 STT/번역/TTS → AI 워커(Python)
학교/학과/과목/교수 CRUD	오디오 track 송수신 → LiveKit + AI 워커
세션(수업) 생성/시작/종료	문서 chunking/embedding/vector 저장 → RAG 서버
LiveKit Room 생성 + 토큰 발급	(Core는 토큰만 주고 빠짐)
파일 업로드 → Blob 저장	
glossary 관리 + Redis prewarm	
RAG 인덱싱 job 트리거만	실제 인덱싱은 RAG 서버가 함

→ **핵심**: Core API는 "오디오를 만지지 않는다." 세션을 켤 때 LiveKit Room을 만들고 토큰을 나눠준 뒤, 실제 처리는 AI 워커/RAG 서버에 **이벤트/트리거로 넘긴다**. 이 경계를 코드 구조에도 그대로 반영합니다.

1. 도메인 모델 (PostgreSQL 테이블)

PDF의 "학교/학과/과목/교수/세션/glossary"를 관계형으로 풀면:

```
School (학교)
├── Department (학과)      school_id FK
│   └── Course (과목)      department_id FK, professor_id FK
│       ├── Session (수업 회차)  course_id FK
│       ├── Material (강의자료)  course_id FK
│       └── GlossaryTerm (용어)  course_id FK

User (교수/관리자) — role: PROFESSOR | ADMIN
```

핵심 엔티티 필드만 추리면:

User	id, email, passwordHash, name, role, departmentId
School	id, name
Department	id, name, schoolId
Course	id, name, code, departmentId, professorId
Session	id, courseId, title, status(SCHEDULED LIVE ENDED), livekitRoomName, supportedLangs(jsonb), startedAt, endedAt
Material	id, courseId, week, sourceType(PDF PPT), fileName, blobUrl, indexStatus(PENDING INDEXING DONE FAILED)
GlossaryTerm	id, courseId, termKo, abbreviation, translations(jsonb: {"zh-CN":..,"vi-VN":..}), pronunciation // pronunciation 은 STT phrase list 와 TTS lexicon 공통 사용

학생은 MVP에서 **계정 없이 QR로 입장** → 굳이 테이블 안 만들어도 됩니다. 세션 입장 시 LiveKit subscribe 토큰만 발급하면 끝. 출석/통계가 필요해지면 그때 SessionParticipant 추가.

2. 전체 폴더 구조 (NestJS)

PDF의 Controller → Service → Repository/DB 구조를 모듈별로 분리합니다.

```

src/
├── main.ts                # 부트스트랩 + Swagger 설정
├── app.module.ts          # 루트 모듈 (전 모듈 import)
├──
├── config/               # 환경설정
│   ├── configuration.ts  # env → 구조화된 config 객체
│   └── env.validation.ts  # Joi 로 필수 env 검증
├──
├── common/               # 횡단 관심사 (cross-cutting)
│   ├── decorators/
│   │   ├── current-user.decorator.ts # @CurrentUser()
│   │   ├── roles.decorator.ts        # @Roles('ADMIN')
│   │   └── public.decorator.ts        # @Public() (인증 제외)
│   ├── guards/
│   │   ├── jwt-auth.guard.ts
│   │   └── roles.guard.ts
│   ├── interceptors/transform.interceptor.ts # 응답 포맷 통일
│   ├── filters/http-exception.filter.ts     # 에러 포맷 통일
│   └── dto/pagination.dto.ts
├──
├── database/
│   ├── database.module.ts # TypeORM 연결
│   └── migrations/
├──
├── modules/              # ★ 도메인 모듈 ★
│   ├── auth/            # 인증 (JWT)
│   │   ├── auth.module.ts
│   │   ├── auth.controller.ts # POST /auth/login, /refresh, GET /me
│   │   ├── auth.service.ts
│   │   ├── strategies/jwt.strategy.ts
│   │   └── dto/login.dto.ts
│   ├──
│   ├── users/           # 교수/관리자
│   ├── schools/         # 학교 CRUD
│   ├── departments/    # 학과 CRUD
│   ├── courses/        # 과목 CRUD
│   ├──
│   ├── sessions/       # ★★★ 수업 세션 (제일 중요) ★★★
│   │   ├── sessions.module.ts
│   │   ├── sessions.controller.ts # 생성/시작/종료/임장토큰
│   │   ├── sessions.service.ts   # start() 오케스트레이션
│   │   ├── entities/session.entity.ts
│   │   └── dto/
│   ├──
│   ├── materials/      # 강의자료 업로드 + 인덱싱 트리거
│   │   ├── materials.controller.ts # POST (multipart)
│   │   └── materials.service.ts
│   ├──
│   └── glossary/       # 용어집 CRUD + Redis prewarm
├──
├── integrations/        # ★ 외부 서비스 어댑터 ★
│   ├── livekit/livekit.service.ts # Room 생성, 토큰 발급
│   ├── blob/blob.service.ts       # Azure Blob 업로드
│   ├── redis/redis.service.ts     # 세션 상태/glossary/pubsub
│   └── rag/rag.service.ts          # 인덱싱 job 트리거 (HTTP or 큐)

```

구조 설계 핵심 2가지:

1. **modules/(도메인) 과 integrations/(외부 연동) 을 분리** — LiveKit·Blob·Redis·RAG 는 "외부 시스템 어댑터"라서 도메인 로직과 섞이면 테스트/교체가 어려워집니다. sessions.service 는 `livekitService.createRoom()` 만 호출하고 LiveKit SDK 세부사항은 모름.

2. 모듈당 **controller/service/entity/dto 4종 세트** — NestJS 표준. `nest g resource modules/courses` 한 줄이면 자동 생성됩니다.

3. 모듈별 책임 + 핵심 코드

3-1. Auth (JWT 인증)

교수 로그인 → access token 발급 → 이후 모든 요청은 JwtAuthGuard 로 보호.

```
// auth.service.ts
async login(dto: LoginDto) {
  const user = await this.userService.findByEmail(dto.email);
  if (!user || !(await bcrypt.compare(dto.password, user.passwordHash)))
    throw new UnauthorizedException('이메일 또는 비밀번호가 틀립니다');

  const payload = { sub: user.id, role: user.role };
  return {
    accessToken: this.jwt.sign(payload, { expiresIn: '1h' }),
    refreshToken: this.jwt.sign(payload, { expiresIn: '14d' }),
  };
}
```

```
// strategies/jwt.strategy.ts
@Injectable()
export class JwtStrategy extends PassportStrategy(Strategy) {
  constructor(config: ConfigService) {
    super({
      jwtFromRequest: ExtractJwt.fromAuthHeaderAsBearerToken(),
      secretOrKey: config.get('jwt.secret'),
    });
  }
  validate(payload: any) {
    return { id: payload.sub, role: payload.role }; // req.user 에 주입
  }
}
```

app.module 에서 JwtAuthGuard 를 **전역 가드**로 걸고, 공개 엔드포인트(로그인, 학생 입장)만 `@Public()` 로 빼는 패턴을 추천합니다. 보안 누락이 줄어듭니다.

중요 — 토큰 2개 구분: 우리 JWT = 우리 API 접근용 (교수 인증). LiveKit 토큰 = LiveKit Room 입장용 (LiveKit SDK가 발급). 둘은 완전히 다른 토큰입니다. 헷갈리지 마세요.

3-2. Sessions (제일 핵심 — 수업 시작 오케스트레이션)

PDF 의 AI 워커 동작("sessionId 받음 → Redis 에서 세션 설정 로드 → glossary prewarm")이 작동하려면, **Core API 의 start() 가 그 사전 준비를 다 해줘야** 합니다.

```
// sessions.service.ts
async start(sessionId: string, user: User) {
  const session = await this.repo.findOneOrFail({
    where: { id: sessionId },
    relations: ['course'],
  });

  // 1) LiveKit Room 생성
  const roomName = `session-${sessionId}`;
  await this.livekit.createRoom(roomName);

  // 2) glossary 를 DB 에서 읽어 Redis 에 prewarm (AI 워커가 여기서 읽음)
  const glossary = await this.glossaryService.findByCourse(session.courseId);
  await this.redis.set(`session:${sessionId}:glossary`,
    JSON.stringify(glossary));

  // 3) 세션 상태 Redis 기록 (지원 언어, roomName 등)
  await this.redis.hset(`session:${sessionId}:state`, {
    status: 'LIVE',
    roomName,
    langs: JSON.stringify(session.supportedLangs),
  });

  // 4) AI 워커에게 "이 세션 시작됐다" 이벤트 발행
  // → 워커가 구독하고 있다가 해당 sessionId 로 Room 입장
  await this.redis.publish('session.started',
    JSON.stringify({ sessionId, roomName }));

  // 5) DB 상태 갱신
  session.status = 'LIVE';
  session.startedAt = new Date();
  await this.repo.save(session);

  // 6) 교수용 LiveKit publish 토큰 + 학생 입장용 QR
  const proToken = await this.livekit.issueToken(roomName, user.id, 'publisher');
  const joinUrl = `https://univoice.app/join/${sessionId}`;
  const qr = await QRCode.toDataURL(joinUrl);

  return { roomName, professorToken: proToken, qr, joinUrl };
}
```

end() 는 역순: LiveKit Room 삭제 → Redis 키 정리(또는 TTL 로 자동) → session.ended 발행 → DB status=ENDED.

컨트롤러 엔드포인트:

POST /courses/:courseId/sessions	수업 회차 생성 (교수)
POST /sessions/:id/start	★ 위 오케스트레이션
POST /sessions/:id/end	종료/정리
POST /sessions/:id/join	학생 입장 → subscribe 토큰 (Public)

학생 입장 토큰은 **subscribe 전용 권한**으로 발급하는 게 포인트:

```
// 학생: 듣기만. 발행 금지
async joinAsStudent(sessionId: string, lang: string) {
  const roomName = `session-${sessionId}`;
  return this.livekit.issueToken(roomName, `student-${randomUUID()}`, 'subscriber');
}
```

3-3. LiveKit 어댑터 (integrations/livekit)

livekit-server-sdk (Node.js) 사용. Room 생성과 토큰 발급 두 가지만 담당.

```
// livekit.service.ts
import { RoomServiceClient, AccessToken } from 'livekit-server-sdk';

@Injectable()
export class LivekitService {
  private client: RoomServiceClient;
  constructor(private config: ConfigService) {
    this.client = new RoomServiceClient(
      config.get('livekit.host'),
      config.get('livekit.apiKey'),
      config.get('livekit.apiSecret'),
    );
  }
  async createRoom(name: string) {
    return this.client.createRoom({ name, emptyTimeout: 60 * 10 });
  }
  async deleteRoom(name: string) {
    return this.client.deleteRoom(name);
  }
  async issueToken(room: string, identity: string,
    role: 'publisher' | 'subscriber') {
    const at = new AccessToken(
      this.config.get('livekit.apiKey'),
      this.config.get('livekit.apiSecret'),
      { identity },
    );
    at.addGrant({
      roomJoin: true,
      room,
      canPublish: role === 'publisher', // 교수만 true
      canSubscribe: true,
      canPublishData: role === 'publisher', // 자막 DataChannel
    });
    return at.toJwt();
  }
}
```

3-4. Materials (업로드 → Blob → 인덱싱 트리거)

PDF 개발 순서 7~9번: file upload → Blob 업로드 → indexing job trigger.

```
// materials.service.ts
async upload(courseId: string, file: Express.Multer.File, week: number) {
  // 1) Azure Blob 업로드
  const blobUrl = await this.blob.upload(file.buffer, file.originalname);

  // 2) 메타데이터 DB 저장 (indexStatus: PENDING)
  const material = await this.repo.save({
    courseId, week,
    fileName: file.originalname,
    sourceType: file.mimetype.includes('pdf') ? 'PDF' : 'PPT',
    blobUrl, indexStatus: 'PENDING',
  });

  // 3) RAG 서버에 인덱싱 트리거만 (실제 처리는 RAG 서버 몫)
  await this.rag.triggerIndexing({
    materialId: material.id, courseId, blobUrl, week,
  });

  return material;
}
```

```
// integrations/rag/rag.service.ts - 두 방식 중 택1
// (A) HTTP 호출 (간단, 동기적 트리거)
async triggerIndexing(payload) {
  await this.http.post(`${this.ragBaseUrl}/index`, payload);
}
// (B) Redis 큐로 던지고 빠지기 (느슨한 결합, 추천)
async triggerIndexing(payload) {
  await this.redis.lpush('rag:index:queue', JSON.stringify(payload));
}
```

Multer 로 @UseInterceptors(FileInterceptor('file')) 쓰면 multipart 처리 끝. Blob 은 @azure/storage-blob 의 BlockBlobClient.uploadData().

3-5. Glossary

과목별 용어 CRUD. translations 는 JSONB 한 컬럼으로 다국어를 묶고, pronunciation 은 STT phrase list 와 TTS lexicon 이 공통으로 씁니다(PDF 명시). 세션 시작 시 SessionsService.start() 가 이걸 Redis 에 prewarm.

```
GET    /courses/:courseId/glossary
POST   /courses/:courseId/glossary      (단건/bulk import)
PATCH /glossary/:id
DELETE /glossary/:id
```

4. 권장 개발 순서 (코드리뷰 마감 고려)

PDF 개발 순서 + 마감을 고려해 데모 가능한 최소 경로부터:

순서	작업	비고
1	nest new + Swagger + PostgreSQL(TypeORM) 연결	30분
2	엔티티 + 마이그레이션 (위 도메인 모델)	DB 설계가 리뷰 핵심
3	Auth(JWT) — login + 전역 JwtAuthGuard	보안 기반
4	Courses CRUD (nest g resource 로 빠르게)	CRUD 패턴 1개 완성
5	Sessions create/start/end + LiveKit 어댑터	★ 데모의 핵심
6	Materials 업로드 + Blob 연동	
7	Glossary CRUD + Redis prewarm	
8	RAG 트리거(큐 push) + Swagger 정리	AI/RAG 팀과 인터페이스 합의

5번(세션 start)에서 LiveKit Room 이 실제로 생기고 교수 토큰이 나오면, 그게 곧 발표용 데모입니다. 거기에 우선 집중하세요.

기술 선택 팁

- **ORM:** TypeORM 추천 (NestJS 와 가장 밀착, Controller→Service→Repository 가 그대로 매핑). DX 우선이면 Prisma 도 OK.
- **Redis 클라이언트:** ioredis (pub/sub + 일반 명령 둘 다 깔끔).
- **env 관리:** @nestjs/config + Joi 검증 — LiveKit/Azure 키가 많아서 누락 방지 필수.
- **팀 인터페이스 합의 먼저:** AI 워커가 Redis 에서 읽는 키 이름(session:{id}:glossary 등)과 RAG 큐 포맷은 다른 팀원과 먼저 맞춰야 나중에 안 깨집니다. 이게 리뷰에서 제일 중요한 부분.